VelaPaw —— 端侧 AI 多宠智能喂食器

技术报告

2026 首届 openVela AI 硬件开发者大赛

赛道：AI 硬件产品创新

队伍名称：CircuitForge（`contest2026_043`）

作品名称：VelaPaw

代码仓库：`contest2026_043_CircuitForge`（分支 `dev-ai-contest-2026`）

硬件平台：openVela（NuttX） / Waveshare ESP32-S3-Touch-LCD-3.5-C

提交日期：2026-09-06（初赛）；决赛更新：2026-11-02

> 本文件为提交技术报告的决赛更新版（2026-11-02）。新增内容以"（决赛更新）"标注，并汇总于 3.7.1；初赛版原样保留。英文版内容相同，见 `VelaPaw_Technical_Report-Finals.pdf` / `.docx`。两版措辞如有出入，以中文版为准。

一、材料清单

材料	要求	文件	说明
作品介绍文档（即本技术报告）	必交	`VelaPaw_技术报告-决赛版.pdf` / `.docx` `VelaPaw_Technical_Report-Finals.pdf` / `.docx`	本文件**即**提交规则要求的"作品介绍文档"——按官网模板填写，未自建格式。提供 PDF 与 DOCX 两种格式，中英双语各一份。决赛更新版（2026-11-02）：新增内容以"（决赛更新）"标注。
演示视频	必交	`VelaPaw_Demo.mp4`	时长 **4 分 58 秒**（≤5 min）；1920×1080 / 30 fps，H.264 High + AAC 48 kHz 立体声，93.3 MB，已内嵌**中英双语字幕**。全片为真机实拍：触摸屏交互、识别门控投喂、步进电机投料、语音排程对话，以及

			`ai_agent` 在真机上的 `ask` 与主动推送控制台实录（末段控制台画面故意不加字幕，以免遮挡日志内容）。视频亦已嵌入仓库 README。
作品展示照片	可选	`hardware/preview/`	3D 打印投料器与整机装配渲染/实拍图。
补充产品介绍	仅存于仓库（不在压缩包内）	`VelaPaw_Project_Intro.pdf` `VelaPaw_Project_Intro_zh.pdf`	早期面向非技术读者的产品介绍（中英双语），含市场与价值分析，留在仓库中供参考。**不**随提交压缩包提交——上表的模板文档才是"作品介绍文档"。
海报 / 答辩 PPT	可选	—	初赛未提交。（决赛更新）已准备决赛答辩 PPT（中英文各一份，8 分钟讲解 + 2 分钟问答）。
AI Coding 日志	必交	`logs/KingMbewe/`	由组委会预装的 `contest-log-collector` 自动归集，未经手工编辑；共 17 个 JSONL 文件、78.7 MB、55 255 条事件。（以上为初赛提交时的数字；此后仍持续归集。）

二、作品基本信息

项目	**内容**
作品名称	VelaPaw —— 端侧 AI 多宠智能喂食器
队伍名称	CircuitForge（编号 `contest2026_043`）

团队分工	**Kingsley Mbewe（金斯利）**，GitHub `KingMbewe` —— **独立参赛，本作品全部工作均由本人完成。**按工作面划分：产品定义与结构设计；自研 3D 打印叶轮投料机构与整机搭建；openVela / NuttX 板级移植与驱动适配（ST7796 显示、OV5640 摄像头、ES8311 音频、28BYJ-48 步进电机、PCF85063 RTC、USB CDC-NCM 网络）；Xtensa LX7 的 INT8 算子库移植；识别模型训练与端侧 TFLite-Micro 集成；LVGL 中英双语 UI 与趋势看板；语音排程 KWS 对话；`ai_agent` 集成与自定义 Skill；以及全部文档、演示视频与提交材料。
选题方向	AI 硬件产品创新
硬件载体	Waveshare ESP32-S3-Touch-LCD-3.5-C（ESP32-S3，Xtensa LX7 双核，8 MB PSRAM，16 MB Flash，320×480 ST7796 电容触摸屏，OV5640 摄像头，PCF85063 RTC，ES8311 音频编解码器）+ 自研 3D 打印叶轮投料机构（28BYJ-48 步进电机 / ULN2003）
操作系统	openVela（NuttX），板级配置 `esp32s3-devkit:waveshare_lcd`
落地的 openVela 能力	**图形**（LVGL）、**AI**（`apps/mlearning/tflite-micro` + `ai_agent` 框架）、**多媒体**（NuttX Audio / I²S / ES8311 + `apps/video` 摄像头栈）—— 三类均已使用
唤醒词	**本作品不含常驻语音唤醒功能。**语音排程需先在"编辑宠物"页手动点击开始，再依次语音应答，属于按键触发的受限词识别（KWS），不存在常驻监听的唤醒词。因此大赛规则中"统一唤醒词须为 *你好，openvela* / *Hello，openvela*"的要求对本项目不适用；若后续加入常驻唤醒，将按规则统一词条。
开源许可	Apache License 2.0
代码托管	GitHub — `github.com/open-vela/contest2026_043_CircuitForge`，分支 `dev-ai-contest-2026`

摘要

多宠家庭的定时喂食器分不清食盆前站着的是哪一只，按只定量、按只控重因而无从谈起。VelaPaw 基于 openVela（NuttX）与 Waveshare ESP32-S3-Touch-LCD-3.5-C，用单摄像头在

端侧识别宠物个体，并以"识别 × 排程"双条件门控投喂：仅当认出该宠物、且它当天有一餐到点未喂时才出粮。识别采用开集度量学习而非分类器，模型只输出 128 维 L2 归一化嵌入，新增宠物为纯运行时操作，无需重训练、无需升级固件。实测成果：为 openVela 原本缺失的 Xtensa LX7 移植 INT8 算子库，单次推理由 **6.75 s 降至 1.31 s**；以逻辑分析仪定位屏幕位同步故障后改用 40 MHz 硬件 SPI + GDMA，全屏重绘由 **246 ms 降至 61 ms**；裁剪 20 个未用工具后，`ai_agent` 单轮请求体由 19 391 B 压入 NuttX IOB 池，端到端时延由 **150–300 s 降至 3.6–35.6 s**。图像处理与投喂决策全部在端侧完成，**任何图像都不离开设备**；唯一联网的 `ai_agent` 以自定义 Skill 读取喂食器写入 littlefs 的记录，仅在食欲越过阈值时才主动发声。断网后喂食器行为完全不变。（决赛更新）Agent 现在还能根据一句输入的话修改宠物的饭点（新增工具 `set_meal_schedule`），其回答与告警也会显示在触摸屏上。

三、技术报告

3.1 绪论

3.1.1 背景与问题定义

多宠家庭里，普通定时喂食器有一个结构性缺陷：**它不知道站在食盆前的是哪一只宠物**。到点开盖，先到的那只把所有粮都吃掉，慢的那只挨饿——按只定量、按只控重、按只观察食欲，全部无从谈起。而"按只定量"恰恰是宠物医生给出减重、控病、术后恢复方案时的第一条要求。

市面上已有的"视觉款"喂食器（例如 2026 年 3 月上市的小米米家智能宠物喂食器 2 视觉版，众筹 ¥449 / 零售 ¥549）解决的是"让主人*看得见*食盆"，摄像头的产物是一路视频流；它并不解决"*认出*是哪一只"。这类产品同时把摄像头画面与账号绑定上云——这是宠物主人对家庭摄像头最主要的顾虑来源。

因此 VelaPaw 定义的问题是：

> 在一台 MCU 级设备上，**不联网、不上云、不用手机 App**，仅凭一颗摄像头就分辨出食盆前是哪一只宠物，并只投喂"这一只、此时此刻该吃的那一餐"；同时把每只宠物的进食量长期记录下来，在食欲异常时主动告知主人。

"主动告知"这一层由 openVela 的 `ai_agent` 框架承担，是全系统**唯一**需要联网的部分；喂食器本体在断网状态下功能完全不变。

3.1.2 技术难点

难点	具体表现
①端侧细粒度识别的算力	宠物个体识别属于细粒度识别，需要 MobileNet 级主干 + 度量学习头。在 Xtensa LX7 上用 TFLite-Micro *参考*算子跑该模型需 **≈6.75 s/次**，完全不可用。前期 TFLM 摸底（`docs/TFLM_SPIKE_REPORT.md`，**在 QEMU 模拟器上进行，非 S3 实测**）用 profiler 定量确认了同类 INT8 CNN 的时间分布：**CONV_2D 占单次推理 84% 的时钟**（643 819 / 768 105 ticks），即延迟几乎完全由 INT8 卷积核决定，而 openVela 树内只有 `cmsis-nn`(ARM) 与 `nnlib-hifi4`(HiFi DSP)，**没有面向 LX7 的算子库。**
②"新增一只宠物"不能重训	消费产品不可能让用户为了加一只猫去训练模型。必须放弃 N 分类，改用开集度量学习，且阈值要在真实光照下站得住。
③大模型放不进 MCU 的内存映射	1.44 MB 的身份模型无法作为 `const` 数组内存映射——ESP32-S3 的 DROM0 MMU 窗口远在此之前就饱和。必须从裸 Flash 读入 PSRAM；而 **Flash 读会挂起 CPU cache，而 PSRAM 正是经该 cache 访问的**，读模型的线程栈若落在 PSRAM 会直接死锁整颗 SoC。
④硬件 SPI 屏幕"渲染不出来"	全屏写正常，任何**局部窗口**（`CASET/RASET` 子矩形 + `RAMWR`）永不出图，而同样的字节序列走 GPIO 位翻转完全正常。该问题**历经约 60 轮盲目编译未能解决。**
⑤ai_agent 在 MCU 上的请求体 vs. 网络缓冲	`ai_agent` 默认注册 36 个工具，其 JSON schema 使单次 LLM 请求体达 **19 391 B**，其中 **9 946 B 是工具 schema**；而 NuttX 的 IOB 池仅 `NBUFFERS(64) × BUFSIZE(196) = 12 544 B`，扣除 `THROTTLE(24)` 后可用约 **7 840 B。**请求体远大于可用缓冲，表现为"时好时坏"的挂死，且无超时返回。
⑥Agent 答案"看起来对"但其实是假的	心跳任务会重放自己的历史、`list_dir` 的前缀匹配与本地工具短路会把 `(no files found)` 直接当成最终答案交给用户、`ask` 在第 7 个词处被 `MAX_ARGS` 截断。这些缺陷都**不会报错**，只会让回答变得流畅而错误。

3.1.3 创新点

1. **"识别 × 排程"双条件门控投喂**——只有"认出的是这只宠物"*且*"它的某一餐已到点且今天未投"两个条件同时成立才投料，每餐每天至多一次，且每餐可单独跳过。只靠

识别会把贪吃的那只喂一整天；只靠定时又会喂到碰巧路过的那只。这个"与"门是整个产品的核心逻辑，也是"按只定量"能成立的前提。

2. **开集度量学习替代分类器**——模型只做"通用宠物特征提取器"，把 128×128 帧映射成 L2 归一化的 128 维嵌入；登记 = 存若干张照片的平均嵌入；识别 = 余弦相似度匹配。**新增宠物是纯运行时操作，无需重训、无需固件更新**。
3. **把一个卡了 60 轮的 bug 变成 4× 提速**——用逻辑分析仪把"软件问题"证伪为"面板位同步问题"（详见 3.4.3），修复后顺势把显示从位翻转升级为 40 MHz 硬件 SPI + GDMA，全屏重绘 **246 ms → 61 ms**。
4. **ai_agent 作为"沉默的健康监护"，而不是聊天机器人**——心跳任务定期读取喂食器自己写在 Flash 上的记录，**只在越过阈值时才开口**；一切正常时设备完全沉默。自定义 Skill 明确规定"只能使用文件里的数字，绝不臆造"。
5. **"离线为主、联网为辅"的清晰边界**——所有投喂决策与全部图像处理都在端侧完成，**没有任何图像离开设备**；拔掉网线，喂食器行为完全不变，只是 Agent 层停止工作。这既是隐私主张，也是可靠性主张。
6. **逐"兜"定量的机械设计**——用连续单向旋转的叶轮转子取代翻板：翻板的投放量随料斗余量漂移，而转子每格容积固定，`克数 = 格数 × 每格克数`，使"按只定量"在机械层面也成立。

3.2 系统方案设计

3.2.1 总体架构

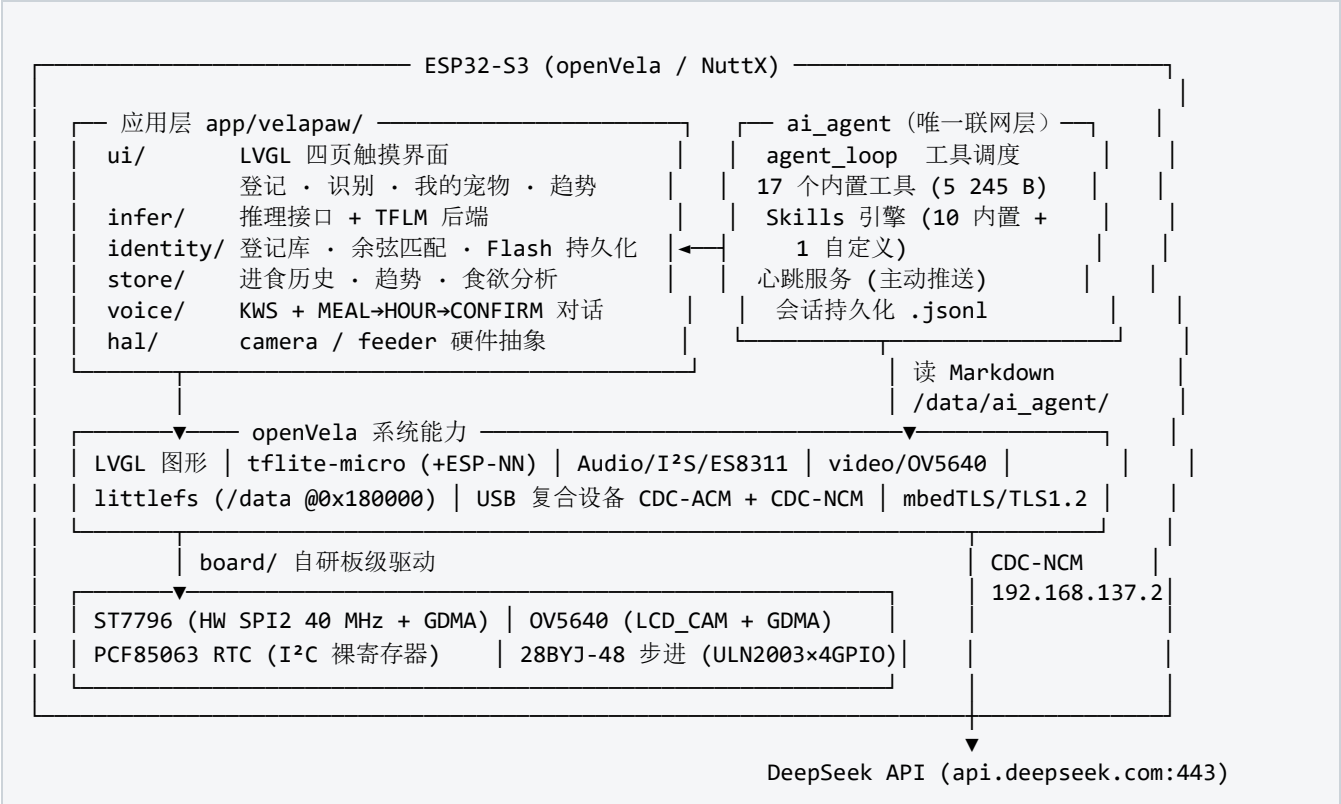


图 3-1　VelaPaw 系统总体框图

3.2.2 方案论证与选型

决策点	最终选型	论证 / 被否方案
识别范式	开集度量学习（余弦匹配）	**否决 N 分类**：每加一只宠物都要重训并 OTA，消费产品不可接受。度量学习把"加宠物"降级为写一条 Flash 记录。代价是对"非宠物物体"的拒识天然较弱，因此把杠杆放在登记质量（近距、光照良好的正脸）而非阈值本身。
推理运行时	TFLite-Micro + ESP-NN	前期用树内 `tflm_benchmark` 在模拟器上摸底，确认 **集成可行、算子全覆盖、张量竞技场仅 84 420 B**（8 MB PSRAM 下可忽略），并定位延迟瓶颈在 CONV_2D（84%）。据此判定必须引入 LX7 SIMD 算子（ESP-NN），否则方案不成立。
模型存放	裸 Flash 分区 + 开机读入 PSRAM	**否决 `const` 数组内存映射**：1.44 MB 远超 DROM0 MMU 窗口。改用 `spi_flash_read`；因 Flash 读会挂起 cache，读取线程的栈必须放在内部 SRAM（`.bss`）而非 PSRAM。
投料机构	28BYJ-48 步进 + 叶轮转子	**否决 MG90S 舵机 + 翻板**：舵机只能往复摆动，进出料口相隔 180° 时净转角为零的摆动永远无法把一"兜"粮送过去，只会投出初始装载量后空转；翻板的投放量还随料斗余量漂移，直接摧毁"按只定量"的产品主张。步进电机连续单向旋转使 `克数 = 格数 × 每格克数` 严格成立，且减速箱低速扭矩大，抗卡粮。
载板	Waveshare ESP32-S3-Touch-LCD-3.5-C	与官方 ESP32-S3-EYE **同一颗 SoC**，openVela 移植与推理工作完全共通；但产品需要**电容触摸屏**（登记 / 编辑排程 / 看趋势）与**硬件 RTC**（断电后排程仍准），EYE 二者皆无。同平台，换更合适的载板。（决赛更新）本板 RTC 未装备用电池，每次拔电都会丢失时间（见 3.4.3）。
板载网络	USB CDC-NCM 复合设备	**否决 RNDIS**：Windows 的 `rndismp6.sys` 硬性要求 RNDIS 位于接口 0，与 CDC-ACM 控制台冲突，实测 Code-10 无法规避。CDC-NCM 使**控制台（COM 口）与网卡（`eth0`）在一根 USB 线上同时可用。否决板载 Wi-Fi 作为演示链路**：Wi-Fi

		已打通并保留在配置中，但演示环境下 USB 复合链路的可复现性更高。
Agent 数据桥	喂食器写 Markdown，Agent 用 `read_file` 读；（决赛更新）改饭点用 `set_meal_schedule`	**否决自定义工具 / IPC / 共享内存**：VelaPaw C 应用每次投喂后把纯文本 Markdown 写到 `/data/ai_agent/velapaw/{status,feed_log}.md`，Agent 用**框架原生的 `read_file`** 读取。零新增工具、零新增协议、零耦合；对 Agent 而言它就是一个普通文件，对 C 应用而言它就是一次 `fprintf`。（决赛更新）读取仍按此方式进行。 Agent 现在唯一能改的东西——宠物的饭点——走一个新增工具 `set_meal_schedule`，它直接调用应用的 `velapaw_meal_set_by_name()`（二者链接在同一个固件镜像中）。仍无 IPC、无共享内存，也没有新增读取工具：`status.md` 现在会列出每只宠物的饭点，每次修改后都重写。
Agent 持久化	littlefs @ `0x180000` 挂载 `/data`	**否决 tmpfs**：API Key、会话历史、Skill 文件每次重启都要重新灌入。littlefs 分区起始于 `0x180000`，而 `write-flash 0x0 nuttx.bin` 只写到约 `0x17ec80`，因此 **`/data` 能跨重启、也跨重新烧录固件保留。**
云端模型	DeepSeek `deepseek-chat`	OpenAI 兼容接口，框架原生支持 function-calling；国内可直连、时延与费用可控。

3.2.3 断网 / 弱网降级策略

这是本作品在方案层刻意设计的一条边界，也是"AI 硬件"与"联网硬件"的分水岭。

功能	断网	说明
宠物识别（INT8 推理）	☑ 正常	模型在 Flash / PSRAM 本地，无任何网络依赖。
识别门控投喂 / 每日上限	☑ 正常	纯本地状态机 + RTC。
体况评估、进食历史与趋势	☑ 正常	第二个端侧模型 + littlefs 本地历史。

语音排程对话（KWS）	☑ 正常	端侧关键词模型，不走网络。
触摸界面全部功能	☑ 正常	LVGL 本地渲染。
Agent `ask` 问答	⛔ 停止	需访问云端 LLM。失败时在控制台给出明确的 `[llm] API error` 行，不产生虚构回答。（决赛更新）用一句话改饭点同样停止；触屏与语音修改仍离线可用。
Agent 主动健康推送	⛔ 静默	心跳任务照常触发、照常读取本地文件，仅 LLM 调用失败；**不会误报，也不会丢数据**——恢复联网后下一次心跳即可基于完整历史给出结论。

弱网处理分三层：（1）传输层——`vela_tls` 维护到 `api.deepseek.com:443` 的连接池并复用（控制台可见 `[vela_tls] Reusing pooled connection`）；空闲 keep-alive 被服务端关闭时会出现零字节响应，框架在带内自动重连，实测 3/3 恢复，**该现象不是缺陷**。（2）超时层——针对 MCU 上的真实往返，把 `AGENT_LLM_TIMEOUT_SEC` 放宽到 **420 s**、socket 超时 **480 s**；此前 60 s 的默认值会把一个已经成功返回的 62.6 s 回答直接丢弃。（3）请求体层——见 3.3.3 的工具裁剪，把请求体压到 IOB 池能承载的范围，这是弱网下最有效的一条。

3.2.4 关键模块设计

模块	位置	职责与关键设计
UI	`app/velapaw/ui/`（`ui_lvgl.c`, 4 104 行）	LVGL 四页界面（登记 / 识别 / 我的宠物 / 趋势）；中英双语字符串表 + 运行时实时切换；趋势页含彩色摄入柱状图、食欲折线与体况仪表。语言为 RAM-only（写 `0x818000` 会锁死 SoC，该地址被列为禁区）。（决赛更新）Agent 的回答与告警以提示条形式显示在铃铛下方；Agent 修改饭点后我的宠物页实时刷新；识别会自动重新就绪；软件卡死监测会监视界面（见 3.5.4）。
推理	`infer/`（`backend_tflm.cc`, 399 行）	两个独立 `MicroInterpreter`（身份 + 体况），张量竞技场置于 PSRAM；**推理跑在专用工作线程**，LVGL 主循环把帧交出去后继续渲染，因此触摸与预览在 1.3 s 推理期间不卡顿；识别为**按需触发**而非常开。

身份	`identity/`（419 行）	登记库、余弦匹配器（匹配阈值 0.45）、逐宠物 Flash 持久化（`0x800000` 魔数 `VPAW`）、宠物头像（`0x810000` 魔数 `VICN`）。
存储/分析	`store/`（`store_stub.c`, 625 行）	30 天摄入窗口、基线与食欲百分比、每日上限、体况记录，并生成给 Agent 用的 `status.md` / `feed_log.md`。**关键修复**：早期"日戳"只在演示按钮下推进，导致历史全部堆进"今天"、摄入量被高估，进而让喂食器*拒绝*投喂；改为 VP02 第 4 个头字 + 时钟驱动的 `vp_sync_day()`，且规定**日戳只增不减**（构建日锚点会让每次冷启动看起来像时间倒退）。（决赛更新）食欲改为按当天计划评分——已出粮克数 ÷ 截至此刻应出粮克数，低于 50% 才告警；基线只统计有过出粮的日子。
语音	`voice/`（`kws.cc` 822 行 + `mic.c` 2 370 行）	14 标签 KWS 模型 + MEAL→HOUR→CONFIRM 受限解码对话；**对原始 softmax 做约束解码，绝不重归一化**；三级门限 0.40 / 0.55 / 0.55，并用投票规则从三次亚阈值读数中救回一次对话。
HAL	`hal/`	摄像头（OV5640 / V4L2 / net / mock 四种后端）与投料（步进 / mock）的抽象，使同一份应用代码可在真机与模拟环境下运行。
Agent 层	`agent/`	1 个自定义 Skill、1 份心跳任务清单、3 个针对 `packages/ai_agent` 的补丁（详见 3.4.5 与 3.3.3）。（决赛更新）另加 `set_meal_schedule`（`tool_meal.c`）、触摸屏提示接入，以及 3.3.3（3）中的 Agent 修复。代码仓库保存的是初赛代码；决赛改动在决赛固件中，并在真机上演示。本表行数为初赛代码的统计。

3.3 核心算法与技术原理

3.3.1 端侧模型

项	身份识别模型	体况评估模型 / KWS
任务	宠物个体识别（开集）	体况三分类（偏瘦/理想/偏胖）；语音关键词识别
结构	MobileNetV3-Small 主干 + 空间注意力 **TSFM** 模块 + ArcFace 度量学习头	紧凑 3 类 CNN；14 标签 KWS 卷积网络
输入 / 输出	128×128 → L2 归一化 **128 维嵌入**	体况：3 类 logits；KWS：14 类 softmax
框架	训练：TensorFlow / Keras（`host/`）；推理：**TensorFlow Lite for Microcontrollers**（`apps/mlearning/tflite-micro`）	
量化	**INT8 训练后量化（PTQ）**，算子全部为 TFLM 内置算子	
模型体积	1.44 MB（Flash `0x600000`）	626 KB（Flash `0x760000`）；KWS 模型内嵌在固件 rodata
算子集	ADD, CONV_2D, DEPTHWISE_CONV_2D, FULLY_CONNECTED, L2_NORMALIZATION, LOGISTIC, MEAN, MUL, PAD（9 个，均为默认 resolver 支持）	
算力占用	单次推理 **≈1.3 s**（ESP-NN SIMD 算子） / **≈6.75 s**（TFLM 参考算子），加速比 **≈5×**（真机实测）。张量竞技场与算子级时间分布来自前期 TFLM 摸底（QEMU，同类 INT8 CNN，**非本模型在 S3 上的实测**）：竞技场 **84 420 B**（非持久 55 296 B + 持久 29 124 B），计算分布 CONV_2D 约 84%、DEPTHWISE_CONV_2D 约 16%。两者的量级足以支撑"竞技场不是约束、卷积核才是瓶颈"的判断，据此确定了优化方向。	

内存放置	两个 `.tflite` 以裸块写入 Flash，开机由 `spi_flash_read` 读入 PSRAM 缓冲；读取线程的栈必须位于内部 SRAM（`.bss`），因为 Flash 读会挂起 PSRAM 所依赖的 cache。

ESP-NN 集成方式与合规说明。1.3 s 这一数字依赖 Espressif 的 ESP-NN（Apache-2.0）SIMD 算子，而该集成修改的是**公共仓** `apps/mlearning/tflite-micro`。按大赛规则，公共仓改动以 fork + PR 形式提交到 `dev-ai-contest-2026`，**不并入队伍仓**。因此本队仓**开箱即用时跑的是参考算子（≈6.75 s）**，套用 ESP-NN fork 后为 ≈1.3 s。集成由三部分组成：（1）`apps/mlearning/esp-nn/` 引入库本体；（2）7 个算子包装（conv / depthwise_conv / fully_connected / pooling / add / mul / softmax）放入 `.../micro/kernels/esp_nn/`；（3）Makefile 块以 `-DESP_NN=1 -DCONFIG_NN_OPTIMIZED -mlongcalls` 编译并**过滤掉对应的参考算子**以避免符号重复。完整包装代码与复现步骤见 `docs/esp-nn/`。

3.3.2 云端模型（Agent 层）

项	**内容**
模型 / 服务商	DeepSeek `deepseek-chat`
接口	OpenAI 兼容的 Chat Completions + **function calling（tools）**；控制台可见 `[llm] OpenAI API with tools (model: deepseek-chat, NNNNN bytes)`
端点与传输	`https://api.deepseek.com:443`，经 `vela_tls` + **mbedTLS，TLS 1.2**；带连接池复用
鉴权	Bearer API Key。**Key 存放在设备 littlefs 的 `/data` 配置文件中，不写入源码、不进仓库。**（开发期曾有一次明文 Key 落入日志的事故，已修复并在归集器中启用 `sk-*` / `ghp_*` / `Bearer *` 的自动脱敏。）
链路	板载 USB **CDC-NCM** 网卡（`eth0`，`192.168.137.2`）→ Windows 主机共享上网 → 公网。同一根 USB 线同时承载 CDC-ACM 控制台。
上行内容	**仅文本**：系统提示 + Skill 摘要 + 工具 schema + 从设备本地 Markdown 读出的数字。**没有任何图像或音频离开设备。**

超时	LLM 420 s / socket 480 s（针对 MCU 侧真实往返整定）

3.3.3 关键机制

（1）识别门控投喂状态机。

```
if  匹配分数 ≥ 0.45  且  匹配到的是宠物 P
and 存在一餐 M 属于 P，其时刻已到、今日未投、未被跳过
and P 今日累计克数 + 本餐克数 ≤ P 的每日上限
then 投料 (克数 / 每格克数) 格 → 记录 feed 事件 → 更新 status.md / feed_log.md
else 界面提示"下一餐 18:00"，不投料
```

（2）工具注册表裁剪——本项目单点收益最大的一处改动。问题的量化形态见 3.1.2⑤。做法是在 `tool_registry.c` 的 `s_excluded_tools` 中排除 20 个与喂食器无关的工具（4 个可穿戴 + 6 个飞书 + 8 个音乐 + 2 个快应用），工具数 36 → **16**，工具 JSON **4 706 B**。效果：可靠性从"时好时坏"变为 **8/8 全通**，端到端时延从 **150–300 s 降到 3.6–35.6 s**。（决赛更新）决赛固件加回了一个我们自己的工具 `set_meal_schedule`：共 17 个工具、工具 JSON 5 245 B；ask 仍在 4–9 s 内完成（见 3.3.4）。

> **连带缺陷（值得写进报告的工程教训）：**裁掉工具会**留下调用方**。这次裁剪孤立了 5 条自然语言快捷路径、1 段手写的 `music_search` 分支，以及每份系统提示里的飞书文案；而 `handle_nl_fast_path()` **运行在 LLM 之前**，因此指向未注册工具的快捷路径会把空白/错误结果**当作最终答案**返回给用户。检查方法固定为改动 `s_excluded_tools` 后 grep 一次被裁工具名，要求返回为空。

（3）三类"静默错误"的修复。它们的共同特征是*答案读起来很通顺，但不是从数据来的。*（决赛更新）下面最后两条是在为决赛加入改饭点功能时发现的；其中第二条恰好相反，是正确的结果被报告为错误：

- **心跳重放自己的历史**——持久化的会话历史跨重启存活，导致第 2 次以后的心跳在"复读"。`history_json` 在循环*外*只 `calloc` 一次，因此需要显式写入 `'\0'`。
- **`list_dir` 前缀匹配 + 本地工具短路**——前缀是按绝对路径匹配的，而模型传的是相对路径；又因该工具位于本地短路集合中，`(no files found)` 被直接当成最终答案交给了主人。修法是**同时接受两种路径写法**（模型实际会用三种）。
- **陈旧的控制台历史抑制工具调用**——修法不是裁剪历史，而是**在消息数组中部插入一条 `system` 消息**强制重新读取；磁盘上的旧历史原样保留，结果仍为 `iters=3 tools=2` 且答案正确。
- **`ask` 在第 7 个词处截断**——根因是 `nsh_commands.c` 的 `MAX_ARGS 8`，已提升到 **32**。
- **本地工具短路截断两步计划（决赛更新）**——改饭点需要两步（先读 `status.md`，再调用 `set_meal_schedule`），但 Agent 循环把这次本地读取直接当作最终答案返回并结束：`status=ok iters=1` 恰恰是失败而非成功。修法是在 `agent_loop.c` 的指令读取例外中加入 `/velapaw/`。

- **修改成功却报错（决赛更新）**——`cJSON_Delete` 释放请求时宠物名仍指向其中，导致发给 LLM 的下一次请求含乱码，被以 HTTP 400 "invalid unicode code point" 拒绝；主人看到 "Sorry, I encountered an error"，而饭点其实已经保存。修法是先复制名字；UTF-8 校验保留作为防护。

（4）验证方法论。本项目对"看起来能用"保持高度不信任，固化了三条判据：`END status=ok iters=N tools=M` 才是真信号（`iters=1 tools=0` 的完美回答等于历史重放）；`[skills] Skills summary: NNN bytes` 才证明 Skill 加载成功（`Skills system ready (N built-in)` 是编译期常量，什么也不证明）；判断回答是否真的来自远端，用 **在运行中途登记一只新宠物**——若下一次触发能发现它，就排除了 RAM 内 `llm_cache` 重放的可能。（决赛更新）对改饭点还有第四条判据：回复文字从不作数，只有 trace 中的 `set_meal_schedule` 调用行或 `status.md` 中的新时间才算。五分钟内重复同一句话可能由 `llm_cache` 直接作答而不调用工具（见 3.7.3）。

3.3.4 openVela 系统能力的深度运用

VelaPaw 不是"跑在 BSP 上的一个薄应用"，而是纵向贯穿了 openVela 的图形、AI、多媒体、文件系统、USB 与网络子系统。三大能力方向**全部落地**：

能力方向	使用的 openVela 组件	使用深度
AI	`apps/mlearning/tflite-micro`（含 `MATH_GEMMLOWP` / `MATH_RUY` / `MATH_KISSFFT` / `SYSTEM_FLATBUFFERS`）	应用直接链接并同时运行**两个独立的 `MicroInterpreter`**（身份 + 体况），竞技场置于 PSRAM，推理在独立工作线程；并把 ESP-NN LX7 SIMD 算子接入该运行时（以 fork/PR 形式提交）。
AI Agent	`packages/ai_agent`（`CONFIG_EXAMPLES_AI_AGENT_VELA`）	真机运行，非模拟器。使用其工具体系、Skills 引擎、心跳（主动）通道、会话持久化与 `vela_tls`；并向框架回馈 **3 个补丁、18 个文件、+570 / −190 行**的修复（见 3.4.5）。（决赛更新）以上数字为初赛提交时的统计；决赛固件另加 `set_meal_schedule` 及 3.3.3（3）的 Agent 修复。
图形	`CONFIG_GRAPHICS_LVGL` + `LV_USE_NUTTX` + `LV_USE_NUTTX_TOUCHSCREEN`、`LCD_FRAMEBUFFER`、`VIDEO_FB`	四页触摸界面；自制 Noto Sans SC 中文子集字库（14/16/20 三档）实现中英双语实时切换；趋势页自绘柱状图 / 折线 / 仪表（LVGL 9.1 无 `lv_meter`，改用 draw_dsc 字段而非 getter）。

多媒体	`CONFIG_AUDIO` + `AUDIO_I2S` + `AUDIO_ES8311` + `ESP32S3_I2S0`、`SYSTEM_NXLOOPER`；`DRIVERS_VIDEO` + `VIDEO`	板载麦克风采集驱动 KWS 语音排程。**向驱动层定位并修复了一个真实缺陷：ES8311 驱动从未编程 RX 的 TDM slot map，这是 RX 通道卡死的根因**；同时发现 `nxlooper` 需要 `CONFIG_AUDIO_DRIVER_SPECIFIC_BUFFERS`。摄像头经 ESP32-S3 LCD_CAM + GDMA 接入 video 栈。（决赛更新）又发现并修复了第二个间歇性麦克风采集故障：`apb_alloc()` 按 D-cache 行对齐音频缓冲区，而未开启 `CONFIG_ARCH_DCACHE` 时该值为 0，PSRAM 缓冲因此只有 8 字节对齐，缓冲地址模 16 余 8 时 RX GDMA 即卡死（18 次卡死全部如此；13 次正常全部余 0）。在 `libs/libc/audio/lib_buffer.c` 中改为至少 64 字节对齐后修复（连续 3 次开机语音均正常）；尚未提交上游。
文件系统	`ESP32S3_SPIFLASH_LITTLEFS`、`FS_PROCFS`、`FS_TMPFS`	`/data` littlefs 分区承载 Agent 配置、Skill、会话与 VelaPaw 的喂食记录，跨重启与跨重烧固件保留。
USB / 网络	`USBDEV_COMPOSITE` + `CDCACM_COMPOSITE` + `CDCNCM_COMPOSITE`、`NET_TCP`、`NETDB_DNSCLIENT`、`CRYPTO_MBEDTLS`	单根 USB 线同时枚举出控制台（CDC-ACM）与网卡（CDC-NCM），使"真机 + 联网 Agent + 可观测控制台"三者同时成立——这是本作品全部 Agent 证据得以录制的前提。（决赛更新）`ask` 在写出请求后卡住的问题追查到 USB FIFO 布局：`ESP32S3_FIFO_SIZE` 为 1280 B（320 字），而 FIFO RAM 只有 256 字（可用 200），CDC-NCM 的 bulk-IN FIFO 因此与 RX FIFO 重叠。将其改为 800，并修复 OTG 端点取消/中止与 CDC-NCM 双缓冲发送后，连续三次 ask 在 4–9 s 内完成，ping 190/190；未单独验证哪一处修改起决定作用；尚未提交上游。
内核 / 内存	`XTENSA_IMEM_USE_SEPARATE_HEAP`、`XTENSA_IMEM_REGION_SIZE=0x48000`、`ARCH_HAVE_EXTRA_HEAPS`、`IOB_NBUFFERS/NCHAINS=64`、`IOB_THROTTLE=24`	见 3.4.5 的四项平台级修复。

3.3.5 对 openVela 的改进建议

以下 10 条均来自本项目真机踩坑。前 6 条于初赛提出，按"对下一位开发者的价值"排序；第 7–10 条为决赛新增（决赛更新）：

1. **为 Xtensa LX7 提供官方的 TFLM 优化算子路径（ESP-NN）。**树内现有 `cmsis-nn`（ARM）与 `nnlib-hifi4`（HiFi DSP），唯独缺少 LX7；而 ESP32-S3 是本届大赛的主力芯片。差距是 **6.75 s → 1.3 s（5×）**，直接决定"端侧视觉 AI 在 openVela 上是否可用"。建议以 Kconfig 开关的形式内置，而非让每支队伍各自 fork。
2. **`mallinfo()` / `free` / `memdump` 在刚启动的板子上会持锁挂死整颗芯片。**本项目中 `ai_agent` 恰好把堆遍历放在 agent 任务的第一条语句，于是每一次 `ask` 都冻结全机（曾一度被误判为 PSRAM 问题）。建议：堆遍历接口增加非阻塞/超时形态，或至少在文档中标注其持锁语义。
3. **`ARCH_HAVE_EXTRA_HEAPS` 是 `select` 符号，永远不出现在 defconfig 中。**而它恰恰是"任务栈不落在 PSRAM"的三个必要符号之一，缺一个就在首次 `ask` 时死锁。建议在 `XTENSA_IMEM_USE_SEPARATE_HEAP` 的帮助文本中显式提示这一依赖。
4. **IOB 池默认容量与 `ai_agent` 的默认请求体不匹配。**默认 36 个工具产生 19 KB 请求体，而默认 IOB 可用约 7.8 KB，症状是无超时的间歇挂死——最难排查的一类故障。建议：`ai_agent` 默认精简工具集，或在 MCU 目标上对请求体大小做启动期断言。
5. **`ai_agent` 的"本地工具短路"应保证与工具注册表一致。**`handle_nl_fast_path()` 早于 LLM 执行，一旦其目标工具被排除，错误结果会作为最终答案返回用户，且无任何告警。建议编译期校验短路表 ⊆ 已注册工具集。
6. **几处"会静默毁掉整机"的配置项值得在 Kconfig 帮助文本中警示：**`CONFIG_RTC`（在本板上会杀死 USB 控制台）、`CONFIG_NETINIT_THREAD`（挂死整机）、`CONFIG_DEBUG_USB_INFO`（塞死控制台）；以及 `CONFIG_NSH_LINELEN` 会在约 80 字符处**静默截断**输入行，用 `echo … >> /long/path` provision 文件时会写出损坏内容而不报错。
7. **USB FIFO 大小应由硬件 FIFO RAM 推导（决赛更新）。**`ESP32S3_FIFO_SIZE` 为 1280 B（320 字），而 ESP32-S3 OTG 的 FIFO RAM 只有 256 字，CDC-NCM 的 bulk-IN FIFO 因此静默地与 RX 重叠。建议按硬件大小推导各 FIFO 的划分，并在初始化时断言放得下。
8. **PSRAM 音频缓冲需满足 DMA 对齐（决赛更新）。**`apb_alloc()` 按 D-cache 行对齐，而关闭 `CONFIG_ARCH_DCACHE` 时该值为 0，缓冲只有 8 字节对齐，ESP32-S3 的 RX GDMA 在地址模 16 余 8 时卡死。建议音频缓冲至少按 64 字节对齐。
9. **内置命令启动失败时 NSH 报 "command not found"（决赛更新）。**内部 RAM 不足时（例如在 `ai_agent` 运行后再执行 `velapaw voice`）任务无法创建，这条提示具有误导性。建议报告真实错误（如 `ENOMEM`）。
10. **`uart_write` 先加锁、后检查 `O_NONBLOCK`（决赛更新）。**因此非阻塞的控制台写入仍可能在 TX 缓冲满时阻塞，长时间运行中曾因此卡住界面；喂食器现在先查询剩余空间（`FIONSPACE`）再打印。建议先检查 `O_NONBLOCK`，缓冲已满时直接返回 `EAGAIN`。

3.4 系统实现

3.4.1 软件 / 固件架构

```
contest2026_043_CircuitForge/
├── app/velapaw/          设备端应用（经 manifest linkfile 映射到 packages/demos/）
│   ├── ui/               LVGL 界面 + i18n + CJK 字库          ui_lvgl.c 4 104 行
│   ├── infer/            推理接口 + TFLM 后端 + 两个 .tflite  backend_tflm.cc 399 行
│   ├── identity/         登记库 · 余弦匹配 · Flash 持久化      419 行
│   ├── store/            进食历史 · 趋势 · 食欲分析            625 行
│   ├── voice/            KWS + 语音对话 + 麦克风采集           kws.cc 822 / mic.c 2 370 行
│   └── hal/              camera(4 后端) / feeder(2 后端)
├── agent/                ai_agent 层：自定义 Skill · 心跳清单 · 3 个框架补丁
├── board/                板文件：显示/摄像头/RTC/步进(esp32s3_st7789.c 961 行)
│                         开机自启(esp32s3_appinit.c 284) · bringup(658) · defconfig(205)
├── host/                 设备外训练与导出（TensorFlow/Keras，约 4 941 行 Python）
├── hardware/             3D 打印投料器（OpenSCAD 源 + STL + BOM）
├── docs/                 工程详解 · 基准测试 · ai_agent 说明（均中英双语）
└── logs/                 AI Coding 会话日志（自动归集，未经手工编辑）
```

手写 C/C++ 代码合计 **13 049 行**（app/ + board/，已排除生成的模型数组、字库表与素材）；主机侧 Python 约 **4 941 行**；对 packages/ai_agent 的补丁 **+570 / −190 行**；板级 defconfig **205 行**。构建产物 nuttx.bin **1 561 920 B（1.49 MB）**。（决赛更新）以上统计针对代码仓库（即初赛代码）；决赛固件由决赛代码树构建，其 nuttx.bin 为 **1 565 456 B**。

Flash 布局（16 MB）

偏移	内容	大小	说明
0x000000	固件 nuttx.bin	1.49 MB	openVela + 应用 + LVGL + TFLM + ai_agent
0x180000	/data（littlefs）	—	Agent 配置 / Key / Skill / 会话 / 喂食记录；跨重烧保留；（决赛更新）另含 fed.bin（当天已出粮的餐次）与 health.log
0x600000	身份模型 velapaw.tflite	1.44 MB	开机读入 PSRAM
0x760000	体况模型 bcs.tflite	626 KB	开机读入 PSRAM

0x800000	已登记宠物（魔数 VPAW）	8 KB	姓名 · 排程 · 面部嵌入
0x810000	宠物头像（魔数 VICN）	32 KB	卡片缩略图

> **禁区：**`0x818000` 一经写入即锁死 SoC，已在工程中列为永久禁用地址（界面语言因此设计为 RAM-only）。

3.4.2 数据流与流程

```
【喂食主流程 —— 全部在端侧，零网络依赖】

 OV5640 帧 (LCD_CAM+GDMA)
       │
       ▼
 预处理 → 128×128 INT8
       │
       ▼  [工作线程，≈1.3 s，UI 不阻塞]
 TFLM 身份模型 ──► 128-D L2 归一化嵌入
       │
       ▼
 与已登记宠物做余弦匹配 ──► (宠物 ID，分数，间距)
       │
       ├── 分数 < 0.45 ──────────────────► 不投料，界面提示"未识别"
       │
       ▼ 分数 ≥ 0.45
 ┌──────────────── 双条件门控 ────────────────┐
 │ ① 该宠物是否有一餐已到点、今日未投、未跳过？ │
 │ ② 今日累计 + 本餐克数 ≤ 每日上限？          │
 └──────┬──────────────────────────┬──────────┘
      都满足                     任一不满足
        │                          │
        ▼                          ▼
 28BYJ-48 步进转 N 格        界面提示"下一餐 18:00"
 克数 = 格数 × 每格克数
        │
        ▼
 写入历史 (littlefs) → 更新趋势 / 食欲基线 / 体况
        │
        ▼
 生成 /data/ai_agent/velapaw/status.md  (逐宠物：今日克数·餐次·7日总量·
                                         基线·食欲%·体况·每日上限·
                                         饭点·今日计划)（决赛更新）
          /data/ai_agent/velapaw/feed_log.md (带时间戳与匹配分的投喂事件)

【Agent 流程 —— 唯一联网的一层】

 (A) 主动：心跳服务每 N 分钟触发
       │  把 /data/ai_agent/HEARTBEAT.md 作为一条 system 消息注入
       ▼
 (B) 被动：控制台 ask "how much did bob eat today"
                或 ask "change bob's dinner to 18:20"（决赛更新）
       │
       ▼
 agent_loop  ──► 系统提示 + Skills 摘要(916 B) + 工具 schema(5 245 B)
```

```
        │
        ▼  TLS 1.2 / mbedTLS / vela_tls  ── CDC-NCM eth0 192.168.137.2 ──► DeepSeek
        │
        ▼ 模型返回 tool_calls
list_dir → read_file(status.md) → read_file(feed_log.md)   [本地执行，不出设备]
或 read_file(status.md) → set_meal_schedule(...)   [决赛更新]
        │
        ▼ 把文件内容回灌，再次请求
最终回答 ──► 控制台 + 触摸屏提示 / 主动推送（红色提示 + 铃铛）
        │        （若全部正常：仅回 HEARTBEAT_OK，设备保持沉默）
        ▼
会话落盘 /data/ai_agent/sessions/tg_<chat_id>.jsonl（console / heartbeat 分命名空间）
```

图 3-2　数据流与处理流程

3.4.3 硬件设计与适配

是否完成全新硬件平台适配 / 驱动开发：是。Waveshare ESP32-S3-Touch-LCD-3.5-C 在 openVela 中**没有现成板级支持**。本项目在 `esp32s3-devkit` 基础上新增了一份完整的板级实现（`board/`，共 1 903 行 C 代码 + 205 行 defconfig），**从零驱动了以下四类外设**：

外设	接法	驱动实现要点
ST7796 触摸屏 320×480	硬件 SPI2 + GDMA，40 MHz；CLK=GPIO5, MOSI=GPIO1, CS=GPIO4；触摸 FT5x06 走 I²C0(SCL=7/SDA=8)	见下方"最硬的一处调试"。像素行需经**内部 SRAM 中转缓冲**——SPI DMA 无法 cache 一致地读 PSRAM；地址窗口设置从 11 次 SPI 调用**合并为 5 次**（LVGL 每次 `putarea` 都会重设窗口）。
OV5640 摄像头	ESP32-S3 **LCD_CAM** 外设 + GDMA 接收通道，SCCB/I²C 配置；经 **30 cm FFC + 1:1 转接板**外引	预览曾出现**90° 转置 + 紫色偏色**，两个独立根因：（a）翻转/镜像位——须清除 `0x3820/0x3821` 的 bit1+bit2，转置本身只能在软件重采样中修正，寄存器结构上做不到；（b）**BLC pattern 寄存器 `0x4514` 须置 `0x88`**。判别方法是一条可复用的规律：**整帧色相是决定性判据——翻转/Bayer 相位错误只会红蓝互换、绝不可能丢掉绿色，因此"紫色（R+B 无 G）"必然是 `0x4514`，不是翻转位。**
PCF85063 RTC	I²C0，**裸寄存器读写**	刻意**不使用 `CONFIG_RTC`**——在本板上它会杀死 USB-CDC 控制台。开机时间种子设计为**以构建日期为下限的地板值**而非"早于 2017 即视为无效"的合理性判断：后者会把板子永久钉在某一天且无法纠正（`wall_set_min` 只改时分，nsh 没有 `date`）。（决赛更新）本板没有备

		用电池，每次拔电都会丢失时间。决赛固件会检测到这一点，从下一天 08:00 开始计时，并在 `health.log` 中记录 `clock lost`；按 RESET 不会丢失时间。
28BYJ-48 步进投料	ULN2003 驱动板，IN1–IN4 = **GPIO9 / 10 / 11 / 43**；5 V 独立供电，**仅共地**	软件走全步序列。**IN4 必须用 GPIO43（U0TXD）而不是 GPIO44（U0RXD）**——GPIO44 被钳位，线圈驱动不干净，这正是早期"电机嗡嗡响但不转"的根因（是引脚问题，不是相序问题）。GPIO43 可用是因为控制台走 USB-Serial/JTAG 而非 UART0。**严禁把步进放在 GPIO12–16**——那是 ES8311 编解码器独占的 I²S 总线（未引到任何排针），已被语音功能占用。
ES8311 音频	I²S0：MCLK=12, BCLK=13, DIN=14, WS=15, DOUT=16；16 kHz	为打通麦克风采集，**定位并修复了驱动层缺陷：从未编程 RX 的 TDM slot map**（RX 通道卡死的真正根因）。另有 PSRAM cache 陷阱迫使采用 18 缓冲池的双 slot 流（活动 slot 每次采集翻转）。（决赛更新）另一个间歇性故障——PSRAM 缓冲地址模 16 余 8 导致 RX GDMA 卡死——通过把音频缓冲按 64 字节对齐来修复（见 3.3.4）。

最硬的一处调试：硬件 SPI 屏幕。症状是全屏写正常、任何局部窗口永不出图，而完全相同的字节序列走位翻转却正常；该问题**熬过了约 60 轮编译**（帧缓冲镜像、DMA 开关、软件 CS、PSRAM 一致性、引脚强制路由等全部无效）。

- **方法——测量，而不是猜。**SPI 引脚没有引到排针，无法直接探测，于是让板级固件通过 GPIO 矩阵把 `FSPID_OUT`/`FSPICLK_OUT` **镜像到空闲排针**，另加一路触发标记，用逻辑分析仪（sigrok/PulseView + FX2）抓取。**决定性的第一步是"地面真值"测试**：先在探针脚上输出 4 路已知方波，读不干净就绝不相信任何采集——这一步立刻暴露了一个接线/接地问题，此前的"乱码"全部源于此；只有在此之后，真正的采集才是可信的。
- **发现。**解码结果显示 ESP32 发出的是**逐字节完全正确**的 `CASET 00 28 00 8B / RASET 00 28 00 8B / RAMWR / pixels`，DC 对齐正确、每字节恰好 8 个干净时钟沿、零毛刺。**外设自始至终是清白的。**
- **根因。**该面板的**片选被硬件拉低**，因此它永远无法借助 CS 边沿重新同步位计数器——它只会一直数时钟沿。固件先用位翻转完成面板初始化，*之后*才把引脚交给 SPI 外设；这次**引脚交接会让时钟多出/少掉一个沿**，从那一刻起面板就错开了一位。此后每条命令都被解码成垃圾，`RAMWR` 永不被识别，没有像素进入 GRAM，屏幕就一直显示位翻转最后画上去的内容——看起来*正好像*"硬件 SPI 什么都没画"。

- **修复。先**把引脚交给 SPI 外设，**再硬件复位面板**（软件复位不行——复位命令自己就会被读错），然后整套初始化全部走 SPI。面板在 SPI 所有权下重新同步之后，局部窗口即正常渲染。
- **收益。**路径打通后，显示彻底脱离位翻转，改用 40 MHz 硬件 SPI2 + GDMA：全屏 480×320 重绘 **≈246 ms → ≈61 ms（4×）**。一个看似死路的 bug 变成了性能提升。

机械设计（`hardware/`）。重力料斗 → 叶轮转子 → 落料槽 → 食盆。全部零件为参数化 OpenSCAD 源码 + 可直接打印的 STL（PLA/PETG，0.2 mm 层高，3 圈外壁，无支撑），含 JLC 代打件清单。摄像头从主板拔下，经 30 cm FFC + 1:1 转接板装在跨过落料槽的 `cam_mast` 双腿支架上的 `cam_pod` 内，使**屏幕朝向主人、镜头朝向宠物**。**安全警示：FFC 反插在上电瞬间是致命的**——板端第 N 脚会对上相机第 25−N 脚，把电源轨直接短到数据/地脚；且该相机模组**内部地未共通**，无法"穿过相机"用万用表确认方向，只能依据丝印 `1/24` 与裸铜连通性核对（已在 2026-07-30 逐点验证）。

3.4.4 应用 / 交互端设计

本产品没有配套 App、没有账号、没有云端控制台——这是产品定位的一部分，而不是功能缺失。交互全部发生在设备本体上：

交互面	设计
触摸屏（主交互）	四页 LVGL 界面：**登记**（拍几张近距正脸照 → 存平均嵌入）· **识别**（实时预览 + 匹配结果 + 投喂）· **我的宠物**（编辑姓名/餐次/克数/每日上限/单餐跳过，删除，头像）· **趋势**（摄入柱状图、食欲折线、体况仪表）。上电**直接进入界面**（`board_app_initialize()` 自启），无需连接串口——它的行为像一台家电，而不是一块开发板。（决赛更新）Agent 的回答与告警现在也显示在屏幕上。
语音（免触摸）	MEAL→HOUR→CONFIRM 三步对话，在"编辑宠物"页点击开始后进行，用端侧 KWS 模型识别受限词表。真机端到端验证通过（例：COMMIT meal 2 = 08:00）。（决赛更新）缓冲对齐修复（见 3.3.4）后，测试的每次开机语音均可用（3/3）。
双语	中 / EN 应用内实时切换，含自制 Noto Sans SC 子集字库；全部界面文案、文档与 README 均为中英双语。
Agent 对话（控制台）	`ask <自然语言>`，例：`ask how much did bob eat today`。回答来自设备本地记录，非预置文案。（决赛更新）一句输入的话也能修改饭点，例：`ask change bob's dinner to 18:20`（`END status=ok iters=3 tools=2`，约 5 s，已存入 Flash）。每次 ask 都会

	在屏幕上以提示条显示：先是 "You: …"，再是 "Agent working… N s"，最后是回答，在铃铛下方停留 30 s。
Agent 主动推送	心跳触发，**无人值守**；只在越阈值时开口。这是"不需要主人走到屏幕前"的那一层。（决赛更新）心跳告警现在也会在屏幕上显示为常驻的红色提示条，同时铃铛变红，并列入铃铛面板。

3.4.5 自定义 Skill（AI 硬件产品创新赛道硬性要求）

Skill 文件：`velapaw-feeding-digest.md`（喂食简报）
仓库位置：`agent/skills/velapaw-feeding-digest.md`
运行时路径：`/data/ai_agent/skills/velapaw-feeding-digest.md`

> **路径说明：**赛道指南给出的运行时 Skill 目录为 `/data/agent/skills/`；本实现使用的是 **`/data/ai_agent/skills/`**，因为这是当前 `packages/ai_agent` 框架在设备上*实际*扫描的目录（控制台日志 `[skills] Skill exists: /data/ai_agent/skills/…` 可证）。二者语义等价，如需对齐规范目录，只是一次挂载点重命名。

Skill 定义（全文）：

```
# VelaPaw Feeding Digest
Report pet feeding, appetite and body condition.

## When to use
When asked about a pet, feeding, how much a pet ate, appetite,
body condition or feeder status. Also used by the periodic heartbeat check.

## How to use
1. read_file /data/ai_agent/velapaw/status.md
   Per pet it gives grams today, meal count, 7-day total, baseline,
   appetite percent, body condition, daily limit.
2. read_file /data/ai_agent/velapaw/feed_log.md
   Recent feed events with timestamps and match score.
3. Report per pet: grams today vs the daily limit, appetite vs baseline,
   and body condition.
4. Flag a concern ONLY if any of these is true:
   - appetite below 50 percent of baseline
   - grams today reached the owner daily limit
   - body condition is not ideal
   - no feed event today in the log
5. Otherwise say the pet is on track, one sentence.
6. Use only numbers from those files. Never invent.
```

触发场景（两条，均已在真机上验证）：

1. **被动触发 —— 主人提问。**控制台输入 `ask how is jay doing today` 一类问题时命中 *When to use*，Agent 依次 `list_dir` → `read_file(status.md)` → `read_file(feed_log.md)`，据实际数字作答。真机实录：`[llm] OpenAI API with tools (model: deepseek-chat, 11559 bytes)` → `END status=ok iters=3 tools=2 llm_ms=4340 elapsed=5s`，回答内容为

"*Today: 15 g in 1 meal / Appetite: 34% of baseline (44 g/day) / Owner daily limit: 60 g/day*"。

2. **主动触发 —— 心跳健康巡检（本赛道要求的"主动 + 执行"场景）。** `/data/ai_agent/HEARTBEAT.md` 中的任务项为：*"Run the VelaPaw Feeding Digest skill. If it finds a concern, notify the owner with numbers. If all is on track, reply HEARTBEAT_OK only."* 心跳服务（**用的是 heartbeat 服务，而非 cron**）按固定间隔把该文件作为一条 `system` 消息注入会话并触发一轮完整的 agent loop。**沉默是正常状态**——只有越过 Skill 第 4 条的四个阈值之一才会推送。已捕获的**无人值守**实录：`iters=4 tools=3`，4 次 `read_file`，请求体依次 **15 592 → 16 209 → 17 697 → 18 760 B**。

为证明"这是真的在跑，而不是重放"，采用了一条可复现的判据：**在心跳运行过程中登记一只新宠物**，若下一次触发能够发现它，则排除 RAM 内 `llm_cache` 重放的可能。该测试已通过——每一次触发都是真实的远端调用。（此前确实存在"第 2 次以后的触发被缓存重放"的缺陷，已通过按 `chat_id` 绕过缓存修复。）

四项平台级修复（`agent/patches/`，3 个补丁 / 18 个文件 / +570 −190 行）——没有它们，`ai_agent` 在这块板子上根本跑不起来：

#	问题	修复
1	`ask` 第一次调用即**冻结整颗芯片**	根因：`agent_loop` 的 pthread 栈落在 **PSRAM（`0x3c…`）**，而 littlefs 的 Flash 读会关闭 PSRAM 所依赖的 cache。修复为三个 Kconfig 符号：`XTENSA_IMEM_USE_SEPARATE_HEAP` + `XTENSA_IMEM_REGION_SIZE=0x48000` + 极易遗漏的 `ARCH_HAVE_EXTRA_HEAPS`。**`0x30000` 只够止住冻结但不够实际出货**——所有任务与 pthread 栈都从这个堆里切，ai_agent 的任务加 6 个 pthread 再叠上 velapaw 的 48 KB 栈会让 `pthread_create` 返回 `ENOMEM`。
2	堆遍历杀死刚启动的板子	`agent_loop_task` 的第一条语句就是 `mallinfo()`，而 `free/memdump/mallinfo` 在刚启动的板子上会持堆锁挂死整机。改为不遍历堆的状态上报。（那个"676 MB 幽灵节点"是坏掉的遍历本身，不是内存损坏。）
3	IOB 耗尽导致请求无声挂死	`IOB_NBUFFERS` / `IOB_NCHAINS = 64`、`IOB_THROTTLE = 24`（**160 条链虽然更大但无法启动**），并需主机先 ping 板子把 ARP 打通。1420 vs 1514 的 MTU 差异**真实存在但不是本故障的原因**，不应去"修" `PKTSIZE`。

4	LLM 超时把成功的回答丢掉	`AGENT_LLM_TIMEOUT_SEC=420`、socket `480`。原 60 s 默认值会丢弃一次已成功返回的 62.6 s 回答，表面症状却是"请求超时"。

3.5 系统测试与结果分析

3.5.1 测试环境

项	配置
被测硬件	Waveshare ESP32-S3-Touch-LCD-3.5-C（ESP32-S3-WROOM-1-N16R8：8 MB PSRAM Octal @80 MHz，16 MB Flash）+ OV5640（30 cm FFC）+ 28BYJ-48/ULN2003（独立 5 V ≥2 A 供电，共地）+ 3D 打印投料机构
固件	openVela / NuttX，`esp32s3-devkit:waveshare_lcd`；`nuttx.bin` 1 561 920 B，为初赛提交时的大小；（决赛更新）决赛固件为 1 565 456 B，于 2026-10-29 烧录
构建环境	VMware 内 Linux 虚拟机，`./build.sh esp32s3-devkit:waveshare_lcd -j4`；Windows 11 主机经 `esptool` 烧录
控制台 / 网络	单根 USB：CDC-ACM 控制台（`nsh>` / `vela>`）+ CDC-NCM 网卡 `eth0` `192.168.137.2/24`，经 Windows 主机 ICS 共享上网
云端	DeepSeek `deepseek-chat`，TLS 1.2 / mbedTLS
主机侧评测	模型可分性评测：验证集 **1 559 张图 / 39 个个体**，INT8 模型，50 次时延采样

3.5.2 功能测试

用例	结果	判据 / 证据
上电自启进入喂食界面	☑	无需连接串口，`board_app_initialize()` 自启；演示视频全程可见

登记新宠物（多张照片 → 平均嵌入）	☑	真机；登记后立即可被识别，**无需重训或重烧固件**
识别并按只投喂	☑	真机；识别 + 排程双条件同时满足才投料
排程门控：未到点不投	☑	界面提示"下一餐 18:00"
每餐每天至多一次 / 单餐跳过	☑	真机
每日克数上限	☑	真机；Agent 侧回读确认"Owner daily limit: 60 g/day"
体况评估（三分类）	☑	真机，第二个 TFLM 解释器
进食趋势 + 食欲骤降告警	☑	趋势页柱状图/折线/仪表；Agent 读出"Appetite: 34% of baseline"（初赛固件；决赛更新：告警改为按当天计划评分，见下方新增行）
步进电机投料（逐格定量）	☑	真机（GPIO9/10/11/43）；机械装配完成
语音排程对话 MEAL→HOUR→CONFIRM	☑	真机端到端（例：COMMIT meal 2 = 08:00）
中英双语实时切换	☑	真机
断电后排程 / 宠物 / 头像保留	☑	Flash 持久化。（决赛更新）时间不保留：无备用电池，拔电即丢失时间，开机后从下一天 08:00 开始并记录 `clock lost`。
断网时喂食器全功能可用	☑	识别 / 排程 / 投料 / 界面 / 语音均不依赖网络
`ai_agent` 在真机启动并加载 Skill	☑	`[tools] Tools JSON built (16 builtin + 0 providers)`、`[skills] Skill exists:`

		`/data/ai_agent/skills/…、[agent] Tools JSON loaded: 4706 bytes`（初赛固件；决赛更新：`17 builtin、5245 bytes`）
`ask` 基于真实设备数据作答	☑	`END status=ok iters=3 tools=2`（`iters=1 tools=0` 即判为重放，不计通过）
主动推送**无人值守**触发	☑	已捕获：`iters=4 tools=3`，4 次 `read_file`
主动推送"一切正常时保持沉默"	☑	阈值未越时仅回 `HEARTBEAT_OK`，不打扰主人
Skill 拒绝臆造数字	☑	Skill 第 6 条 + 回答中的数字与 `status.md` 逐项一致
一句话修改饭点（决赛更新）	☑	`ask change bob's dinner to 18:20` → `END status=ok iters=3 tools=2`，约 5 s；新时间出现在 `status.md` 中并已存入 Flash
Agent 回答与告警显示在触摸屏上（决赛更新）	☑	提示条依次显示 "You: …" → "Agent working… N s" → 回答；心跳告警显示红色提示条并使铃铛变红；我的宠物页实时刷新
食欲对照当天计划（决赛更新）	☑	`today's plan: 39 g`；0 g 时按 Next Day → `ALERT … only 0% of its plan, under half`，铃铛变红，"Appetite drop (0%)"；RESET → 恢复灰色；出粮一餐后 → `appetite today: 100% of the 13 g due so far`
RESET 后不会重复出粮（决赛更新）	☑	电机转动前先写入 `fed.bin`；RESET 后 Bob 被识别，屏幕显示 "Next meal 12:15"，未出粮
拔电后从新的一天开始（决赛更新）	☑	`health.log` 中记录 `08:00 boot rst 0x01 clock lost`；早餐照常出粮；无误报食欲告警

每次开机语音对话可用（决赛更新）	☑	3/3 次开机：`eof 193`、`capfail 0`、`COMMIT meal 1 = 09:00`
连续三次 ask（决赛更新）	☑	9 s / 4 s / 5 s；ping 190/190

3.5.3 性能测试

指标	**实测**	**对照**	**说明**
身份模型单次推理时延	**≈1 310 ms**	≈6 750 ms（TFLM 参考算子）	ESP-NN LX7 SIMD 算子，**加速 ≈5×**。推理跑在独立线程，**UI 不阻塞。**
识别帧率（按需模式）	≈0.76 次/s	≈0.15 次/s	识别为宠物到盆时按需触发，非常开推理；这是刻意的功耗与算力设计。
全屏重绘（480×320）	**≈61 ms**	≈246 ms（GPIO 位翻转）	40 MHz 硬件 SPI2 + GDMA，**提速 ≈4×**；窗口设置由 11 次 SPI 调用合并为 5 次。
张量竞技场占用 （前期摸底，QEMU）	**84 420 B**	配置 204 800 B	非持久 55 296 B + 持久 29 124 B。**非 S3 实测值**，但足以判定在 8 MB PSRAM 下竞技场不是本项目的约束。
计算分布（单次推理） （前期摸底，QEMU）	CONV_2D 84% DEPTHWISE 16%	—	643 819 / 122 680 / 768 105 ticks；`AllocateTensors` 24 860 ticks（一次性）。**非 S3 实测值**；用于判定优化必须落在 INT8 卷积核上，该判断随后被 S3 上 5× 的实测加速验证。
固件体积	**1 561 920 B**	16 MB Flash	含 openVela + LVGL + TFLM + `ai_agent` + 应用；两个模型另占 2.05

			MB 裸 Flash。（决赛更新）决赛固件为 1 565 456 B。
Agent 端到端响应	**3.6 – 35.6 s**	**150 – 300 s**（工具裁剪前）	典型实录：`llm_ms=4340 elapsed=5s`、`llm_ms=5260 elapsed=7s`。（决赛更新）每次改饭点约 5 s；USB 修复后连续三次 ask 为 9 / 4 / 5 s。
Agent 请求体大小	11 312 → 12 667 → 14 008 B（3 轮 `ask`）	19 391 B（裁剪前单轮）	工具 schema 由 9 946 B 降至 **4 706 B**；Skills 摘要 916 B。（决赛更新）加入 `set_meal_schedule` 后：17 个工具、5 245 B。
Agent 缓冲配置	ctx 8 192 / hist 8 192 / tool 8 192 B；工具输出池 2 × 16 384 B	—	控制台启动日志可见。
网络缓冲	IOB 64 × 196 = 12 544 B（THROTTLE 24 后可用 ≈7 840 B）	默认值不足	这一比值是 3.1.2⑤ 那个"间歇挂死"的量化根因。
主机侧模型时延	fast 0.5 ms / accurate 0.6 ms（p95 0.6 / 0.7 ms）	—	**主机 CPU 相对值，不代表 ESP32-S3**；仅用于 A/B 比较。

3.5.4 可靠性与准确性

（a）识别可分性（主机侧，验证集 1 559 张图 / 39 个个体，INT8）

指标	fast 模型	accurate 模型（已选用）
模型体积	1 312 KB	**1 403 KB**
算子数	8	9
类内余弦相似度	0.346	0.347
类间余弦相似度	0.103	**0.023**
可分间距	0.243	**0.324**
EER 阈值	0.130	0.070
等错误率 EER	0.328	**0.291**

选用 accurate 模型的依据是**可分间距**（0.324 vs 0.243）与更低的 EER，代价是 +91 KB 体积与 +1 个算子——在 16 MB Flash / 8 MB PSRAM 的预算下这个代价可以忽略。

> **对该结果的诚实解读：**EER 0.291 是在**公开宠物数据集的 39 个个体、开集条件**下测得的*下界式*指标，它衡量的是"任取两张图判断是否同一只"的难度，而**不是产品实际场景的准确率**。真实场景要容易得多：家庭中通常只有 2–4 只宠物、登记照由主人近距离正脸拍摄、匹配阈值取 0.45（高于 EER 阈值，偏向"宁可拒识、不可误投"）。误报/漏报的主导因素**不是阈值，而是登记质量**——这是开集度量学习的结构性特征：通用特征提取器天然难以拒绝任意非宠物物体，因此产品把杠杆放在引导用户拍出紧凑、光照良好的正脸照。**尚未完成的量化工作：**在最终相机姿态（`cam_mast` 支架、当前色彩标定）下针对自家 3 只宠物的误报率 / 漏报率长期统计。当前 3 只宠物的嵌入是在**存在紫色偏色的固件版本**下采集的，`0x4514` 修复后需要重新登记，此前的现场准确率数据不具备可比性，因此本报告**不给出**该数字，而不是给一个不可信的数字。

（b）Agent 可靠性

指标	工具裁剪前	裁剪后	说明
连续 ask 成功率	间歇性失败（无超时挂死）	**8 / 8**	判据是 END status=ok 行，不是回答读起来是否通顺
端到端时延	150 – 300 s	**3.6 – 35.6 s**	—
TLS 连接中断恢复	**3 / 3 带内恢复**		空闲 keep-alive 被服务端关闭 → 零字节响应 → vela_tls 自动重连。**此现象经确认不是缺陷。**
主动推送真实性	**每次触发均为真实远端调用**		用"运行中途登记新宠物、看下一次触发能否发现"证伪缓存重放

（c）异常恢复与稳定性

- **断电恢复：**littlefs 保宠物 / 排程 / 头像 / 进食历史 / Agent 会话与 Key。/data 起始于 0x180000，而固件只写到约 0x17ec80，**因此重新烧录固件也不会丢数据**。时间不会保留（决赛更新）：本板没有备用电池，每次拔电都会丢失时间。决赛固件随后从下一天 08:00 开始，并在 health.log 中记录 clock lost；按 RESET 不丢时间。白天中途重新上电可能再次出粮，但不会漏喂。
- **至多出粮一次（决赛更新）：**每次出粮之前，喂食器先把该餐记入 fed.bin（魔数 VPF1），因此按 RESET 不会重复出粮。已验证：RESET 后 Bob 被识别，屏幕显示 "Next meal 12:15"，没有出粮。
- **网络异常：**见 3.2.3 降级表。LLM 失败时输出明确的 [llm] API error NNN:，**不产生虚构回答**。
- **内存波动：**本项目**刻意不报告运行时堆余量数字**——原因见 3.4.5 修复 #2：在这块板子上堆遍历接口会持锁杀死整机，修复方式是改为不遍历堆的状态上报，因此固件打印的堆数值是**假定值而非实测值**，作为量化指标引用会构成误导。真正约束内存的是编译期的 XTENSA_IMEM_REGION_SIZE：0x30000 时 pthread_create 返回 ENOMEM，0x48000 可稳定承载 ai_agent 任务 + 6 个 pthread + velapaw 的 48 KB 栈——**这个"能/不能创建线程"的二值结果，比一个不可信的堆余量数字更有信息量**。
- **连续运行时长：**已完成的最长受控观测为演示录制中的 **600 s（sleep 600）** 无人值守区间，期间心跳按期触发、控制台无异常、Agent 正常应答。**更长时间的老化测试（**

24 h / 7 d）尚未进行，列入 3.7.3 的后续工作，本报告不做外推。（决赛更新）`health.log` 现在每 10 分钟记录一行状态；决赛固件的整夜运行测试尚未完成，结果将记录在其中。

- **长时间运行修复与软件卡死监测（决赛更新）：**识别会自动重新就绪（点击屏幕、画面变化，或某餐到点后的前 30 分钟内每分钟一次）；TX 缓冲已满时跳过控制台打印（`FIONSPACE`）；并修复了跨午夜与跳餐相关的缺陷。软件卡死监测会监视界面（2 分钟；出粮期间 10 分钟）与识别工作线程（1 分钟），发现卡死时向 `health.log` 写入 `STALL` 并复位整板。它只是软件层面的监测，无法发现关中断状态下的卡死；复位路径尚未在真实卡死中触发过。
- **功耗：尚未使用功率分析仪做逐工况测量**，因此不给出数字。设计上的低功耗依据是"识别按需触发而非常开推理"（避免 1.3 s 级推理连续占用双核）与"投料仅在门控通过时短时驱动步进"；步进电机 ≈240 mA/相由独立 5 V 供电承担，不经过主板。定量测量列入后续工作。

3.5.5 结果分析

把测试结果放回 3.1.2 的六个难点：

1. **难点①（算力）已解决且量化：**6.75 s → 1.31 s。关键在于前期用 profiler 先确定"84% 的时间在 CONV_2D"，才把全部工程投入压在算子库上，而不是四处调参。**先测量、后优化**是本项目最有回报的一条方法论。
2. **难点②（不重训）已解决：**登记是纯运行时操作，真机验证通过。代价是拒识能力弱于闭集分类器，这一点在 3.5.4 中如实披露而非回避。
3. **难点③（大模型加载）已解决：**裸 Flash + PSRAM，附带产出了"Flash 读会挂 cache"这条对任何 ESP32-S3 + PSRAM + 大模型项目都通用的经验。
4. **难点④（SPI 屏幕）已解决并转化为 4× 收益：**60 轮盲试的失败与一次逻辑分析仪测量的成功，其差别不在努力程度，而在于是否**把猜测换成了观测**。
5. **难点⑤（请求体 vs IOB）已解决且是单点收益最大项：**可靠性"时好时坏 → 8/8"，时延 150–300 s → 3.6–35.6 s，仅靠删掉 20 个用不上的工具。这条经验对所有在 MCU 上跑 `ai_agent` 的团队都适用。
6. **难点⑥（静默错误）已解决，并沉淀为判据：**本项目最重要的产出之一是三条"读日志而不是读文案"的验收判据（`END status=` / `Skills summary: NNN bytes` / 运行中途注入新数据）。它们是把"看起来能用"和"确实在用"分开的工具。

3.6 AI-Native 开发说明

本项目**全程采用 AI 辅助开发**，会话日志由组委会预装的 `contest-log-collector` 自动归集到 `logs/KingMbewe/`，**未经任何手工编辑**。以下数据均由脚本从该日志目录与本机原始会话记录

直接统计得出。（决赛更新）这些数字为初赛提交时的统计（日志截至 2026-09-04），此后仍在持续归集。

项目	数据
AI Coding 代码占比	**≈100%**（按"经由 AI 会话产出/修改"口径）。仓库中手写 C/C++ **13 049 行**（`app/` + `board/`，已排除生成的模型数组、CJK 字库表与素材）、主机侧 Python **约 4 941 行**、对 `packages/ai_agent` 的补丁 **+570 / −190 行**、板级 defconfig **205 行**，合计约 **18 800 行**，全部在 AI 结对会话中产出并逐次在真机上验证。人类承担的部分是：硬件搭建与接线、逻辑分析仪 / 万用表实测、每一轮真机烧录与现象判读、以及方向决策与对 AI 结论的证伪——本报告中多处"某某说法已被推翻"的记录，正是这道人工核验环节的产物。
使用的 AI 工具	**Claude Code**（CLI + VS Code 扩展）为唯一编码工具。会话中出现的模型：`claude-opus-5`、`claude-opus-4-8`、`claude-sonnet-5`、`claude-fable-5`。 日志规模：**17 个 JSONL 文件 / 78.7 MB / 55 255 条归集事件**，其中 user 7 559 条、assistant 14 963 条、tool 32 733 条，跨 **2026-07-08 → 2026-09-04** 共 15 个归集日期目录。 工具调用分布（前列）：`Bash` 5 272、`PowerShell` 4 340、`Edit` 2 436、`Read` 2 263、`Grep` 713、`Write` 458、`AskUserQuestion` 247、`TodoWrite` 168、`ToolSearch` 126、`WebFetch` 89、`Glob` 79、`Artifact` 38、`ScheduleWakeup` 34、`WebSearch` 29、`Monitor` 22、`Skill` 17。`Bash` 与 `PowerShell` 合计 9 612 次的占比，直接反映了这是一个**以"编译—烧录—在真机上观察"为主循环**的硬件项目，而非纯软件项目。
MCP 工具使用情况	**设备端：未使用。**`CONFIG_AI_AGENT_MCP` 在板级 defconfig 中**显式关闭**（连同 Feishu / WeChat / MQTT / Node），这是一个刻意的取舍——见 3.3.3，请求体大小是本项目在 MCU 上最硬的约束，任何非必要的工具/协议扩展都会挤占 IOB 预算。 **开发端：配置了 2 个 MCP 连接器**（claude.ai 连接器、Google Drive），用于设计稿与数据集资料的读取；因需要交互式 OAuth 授权，在本项目的自动化会话中**未实际调用。** 因此本项目的 MCP 使用量诚实记为 **0 次调用。**
Skills 使用与新增情况	**新增运行时 Skill：1 个** —— `velapaw-feeding-digest`（`agent/skills/velapaw-feeding-digest.md` → 板上 `/data/ai_agent/skills/`），满足本赛道"≥1 个自定义 Skill"的硬性要求，定义与触发场景见 3.4.5。设备上另有框架自带的 **10 个内置 Skill**，系统提示中的 Skills 摘要为 **916 B。**

	开发侧 Skill：使用了组委会经 `.claude` manifest 预装的 Skill 集合，其中在本项目中实际发挥作用的包括 `contest-log-collector`（AI Coding 日志归集）、`kconfig-tweak`（免 menuconfig 改 `.config`）、`openvela-build`（构建流程）、`submit-pr`（提交流程），以及通用的 `artifact-design`。`Skill` 工具在归集日志中共触发 **17 次**。 **另建持久化记忆库 40+ 条**，用于跨会话保存"哪些做法已被真机证伪"的结论（例如"CONFIG_RTC 会杀死 USB 控制台"、"`0x818000` 会锁死 SoC"、"重复 wapi 挂控制台一说已撤回"），显著降低了长周期项目中重复踩坑的成本。
Token 使用总量	统计自本机 35 074 条带 usage 记录的助手消息： 输入 token：**340 649** 输出 token：**44 262 781** 缓存写入 token：**181 608 127** 缓存读取 token：**6 725 913 276** **合计：6 952 124 833（约 69.5 亿）** 其中不含缓存读取的部分为 **226 211 557（约 2.26 亿）**。缓存读取占 96.7%，是长上下文硬件调试项目的典型形态——每一轮"改配置 → 编译 → 烧录 → 读控制台"都要携带完整的板级上下文重新推理。

AI 在本项目中承担的角色（按贡献排序）：

1. **把"看起来是软件 bug"的问题重新定义为可测量的物理问题。**硬件 SPI 那 60 轮失败的编译都是在软件里找原因；转折点是决定"镜像 SPI 引脚到排针 + 先做地面真值测试 + 用逻辑分析仪解码"。这个方案的设计与逐步排障是人机协作的产物，而结论——*外设是清白的，问题在面板的位同步*——只有靠观测才能得到。
2. **在大量相似症状中做判别式区分。**例如"整帧色相是决定性判据：翻转/Bayer 相位错误绝不可能丢掉绿色，所以紫色必然是 `0x4514` 而非翻转位"，把一个原本要盲试几十个寄存器的问题一次定位。
3. **建立并执行验伪判据。**本项目多条早期"结论"最终被推翻并在记忆库中显式标注为撤回（如"重复 wapi 会挂控制台"实为测试脚手架假象、"陈旧历史抑制工具调用"由一次受混淆的 A/B 得出、"死 TX ⇒ 死时钟"在寄存器层面被证伪）。**把被推翻的结论记录下来，与记录正确结论同等重要。**
4. **跨越软硬件边界的排障。**ES8311 的 RX TDM slot map 缺失、IOB 池与请求体的比值、PSRAM 栈与 Flash cache 的互斥——这些都不在单一层次内，需要同时握住内核配置、驱动寄存器与应用行为。

3.7 总结与展望

3.7.1 成果总结

VelaPaw 在一颗 ESP32-S3 上、在 openVela 之上，完整实现并**在真机上验证**了一台多宠智能喂食器的全部核心能力：

- **端侧个体识别 + 双条件门控投喂**——开集度量学习，新增宠物无需重训；识别与排程同时成立才投料。
- **端侧健康监测**——体况三分类、30 天摄入趋势、食欲骤降告警。
- **免触摸语音排程**——端侧 KWS 驱动的三步对话，真机端到端验证。
- **端侧 AI Agent**——`ai_agent` 框架真机运行，1 个自定义 Skill，`ask` 被动问答 + 心跳主动推送，**已捕获无人值守自动触发**；（决赛更新）决赛版还能根据一句话修改宠物饭点，并在触摸屏上显示回答与告警。
- **全新硬件平台适配**——从零完成 ST7796 显示（40 MHz HW SPI + GDMA）、OV5640（LCD_CAM + GDMA）、PCF85063 RTC、28BYJ-48 步进四类驱动，并向驱动层回馈了 ES8311 RX slot map 的真实缺陷修复。
- **可制造的机械设计**——参数化 OpenSCAD 投料器 + STL + BOM，逐格定量使"按只定量"在机械层面成立。
- **对 openVela 框架的回馈**——3 个补丁 / 18 个文件 / +570 −190 行，另有 ESP-NN 集成以 fork+PR 形式提交。

关键量化成果：推理 **6.75 s → 1.31 s（5×）**；UI 重绘 **246 ms → 61 ms（4×）**；Agent 响应 **150–300 s → 3.6–35.6 s**，可靠性 **8/8**。

初赛之后新增（2026-09-06 → 2026-11-02），均已在真机上验证（决赛更新）：

- **Agent 修改饭点**——新增一个工具 `set_meal_schedule`；输入一句话（如 `ask change bob's dinner to 18:20`）即可修改时间并存入 Flash（`iters=3 tools=2`，约 5 s）。
- **Agent 上屏**——每次提问及其回答以提示条显示在铃铛下方；心跳告警显示红色提示条并使铃铛变红；我的宠物页实时刷新。
- **食欲按当天计划评分**——已出粮克数 ÷ 截至此刻应出粮克数，低于 50% 才告警；基线只统计有出粮的日子。按计划出粮满一天现在显示 100%（以前为 65%），拔电也不再误报。
- **稳定性**——识别自动重新就绪，控制台打印不再阻塞，`fed.bin` 保证 RESET 前后至多出粮一次，软件卡死监测写入 `health.log` 后复位整板，时间丢失后从下一天 08:00 开始。
- **USB 网络修复**——消除了导致 ask 卡住的 USB FIFO 重叠（`ESP32S3_FIFO_SIZE` 1280 → 800，另有 OTG 与 CDC-NCM 修复）：连续三次 ask 为 9 / 4 / 5 s，ping 190/190。
- **麦克风修复**——音频缓冲按 64 字节对齐，消除了间歇性采集卡死：连续 3 次开机语音均正常。

以上改动在决赛固件中，并在真机上演示；代码仓库保存的是初赛代码，两项驱动修复尚未提交上游。

3.7.2 应用前景与商业价值

市场信号。2026 年 3 月，小米发布米家智能宠物喂食器 2 视觉版（众筹 ¥449 / 零售 ¥549）——一款专门为把摄像头引入喂食场景而生的量产产品。这等于是最了解宠物主人付费意愿的公司，用真金白银独立验证了 VelaPaw 立项时的核心判断：**摄像头理应长在喂食器上**。VelaPaw 在同一判断上更进一步：从"看到食盆"到**"认出是哪一只"**，且全部端侧完成。

目标受众（按付费意愿排序）：

1. **多宠家庭（核心）**——痛点最尖锐：定量管理在多宠环境下完全失效。这是唯一"没有个体识别就无法解决"的人群。
2. **处方饮食 / 减重管理的宠物主人**——兽医给出的克数方案需要被真实执行并留下记录；VelaPaw 的每日上限 + 逐只历史正是这份"执行凭证"。
3. **隐私敏感人群**——不愿家中摄像头画面进入任何云端账号。VelaPaw 的"图像永不出设备"是硬承诺而非隐私政策条款。
4. **老年宠物看护者**——食欲骤降是许多疾病的最早信号；主动推送把"需要主人天天记录"变成"设备发现异常才开口"。

商业模式。核心是**一次性硬件销售，零订阅**——这是端侧 AI 直接带来的结构性优势：没有云推理成本，就不需要靠订阅费摊薄。

收入来源	**说明**
整机销售（主）	对标视觉款喂食器 ¥449–549 价位带。BOM 主体为 ESP32-S3 模组 + 3.5″ 触摸屏 + OV5640 + 步进 + 注塑件，规模化后成本结构与现有视觉款接近，而**省掉了云端推理与存储的长期成本。**
耗材与配件	可洗料仓 / 不同容积转子（改一个 `WIDTH` 参数即改变每格克数）/ 备用食盆。
可选云增值（不绑定）	Agent 层的健康简报可做成可选订阅（长期趋势报告、兽医可读的导出）。**关闭订阅后设备全部核心功能不受影响**——这与"停订阅就变砖"的竞品形成明确差异。
B 端延伸	宠物医院 / 寄养机构的多笼位定量投喂与进食记录，本质是同一套"识别 + 定量 + 留痕"能力。

差异化壁垒：（1）个体识别而非"看得见"；（2）无 App、无账号、无云——安装即用；（3）断网全功能；（4）逐格定量的机械保证；（5）主动健康监护而非被动查看。

3.7.3 不足与未来工作

以下按**诚实优先**列出，包括本报告刻意*未*给出数字的项：

不足	现状与计划
现场识别准确率未量化	现有 3 只宠物的嵌入是在**存在紫色偏色**的固件版本下采集的；`0x4514` 修复后需**全部重新登记**，此前的现场数据不具备可比性。计划：重新登记后，在最终相机姿态下做为期 2 周的逐次匹配记录，给出误报率 / 漏报率与阈值 ROC。
长时老化测试未做	最长受控观测为 600 s 无人值守区间。计划：24 h 与 7 d 连续运行，记录心跳触发成功率、TLS 重连次数、littlefs 写入次数与磨损、以及是否出现缓慢的资源泄漏。（决赛更新）仍未完成：`health.log` 现在每 10 分钟记录一行状态，决赛固件的整夜运行将留下记录。
功耗未测量	尚无功率分析仪逐工况数据。计划：分别测量待机（屏幕常亮 / 息屏）、单次推理、投料三种工况的平均与峰值电流。
运行时堆余量不可观测	因堆遍历接口会挂死本板，当前状态上报为假定值。计划：实现一个不持锁的轻量堆统计（或按区间采样），使内存波动成为可量化指标——这同时也是给 openVela 的改进建议 #2。
ESP-NN 未内置于队伍仓	按规则以 fork+PR 提交，因此本仓开箱跑的是参考算子（6.75 s）。计划：完成 PR 合入后在 README 中给出 PR 链接，并提供一键套用脚本。
非宠物物体的拒识较弱	开集度量学习的结构性特征。计划：加入一个极轻量的"是否为宠物面部"前置门（二分类），在进入嵌入匹配前先过滤，预期能显著压低误投。
无常驻语音唤醒	当前语音排程需先点击开始。计划：若加入常驻唤醒，将**严格采用大赛统一唤醒词**（*你好，openvela* / *Hello，openvela*）。
Skill 运行时目录与规范不一致	实现使用 `/data/ai_agent/skills/`（框架实际扫描目录），规范为 `/data/agent/skills/`。计划：待框架统一后一次性对齐挂载点。

Agent 提问仍需在控制台输入	当前 `ask` 走 NSH 控制台。计划：把 Agent 的问答与推送接到触摸屏上的一个"健康"页，让主人无需串口即可看到简报。（决赛更新）回答与告警现已显示在触摸屏上；提问仍需通过控制台。
重复同一句话可能跳过工具（决赛更新）	`llm_cache`（RAM 中 8 个槽、5 分钟）可能直接用缓存回答完全相同的句子而不调用 `set_meal_schedule`。修复方案：调用过工具的回合不写入缓存（尚未应用）；在此之前请换一种说法。
时钟没有备用电池（决赛更新）	每次拔电都会丢失时间；开机后从下一天 08:00 开始并记录 `clock lost`。计划：为 RTC 加装备用电池。
决赛改动未进入仓库或上游（决赛更新）	代码仓库保存的是初赛代码；决赛改动（包括 USB FIFO 与音频缓冲修复）只存在于决赛固件中。计划：以补丁形式提交。
未启用硬件 WDT（决赛更新）	软件卡死监测无法发现关中断状态下的卡死。计划：启用 ESP32-S3 的硬件 WDT。
